Microservices Characteristics

Lesson 2 · the 9 characteristics, and when NOT to use microservices

In this lesson → ① Where the definition comes from ② The 9 characteristics ③ Conway's law ④ Distributed monolith ⑤ 5 signs NOT to use ⑥ Modular monolith ⑦ Readiness checklist ⑧ Interview Q&A

Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① Where the definition comes from

> **Microservices** have no single official definition. The reference everyone uses is the **2014 article "Microservices" by James Lewis & Martin Fowler**, which lists 9 common characteristics. They are a pattern, not strict rules.

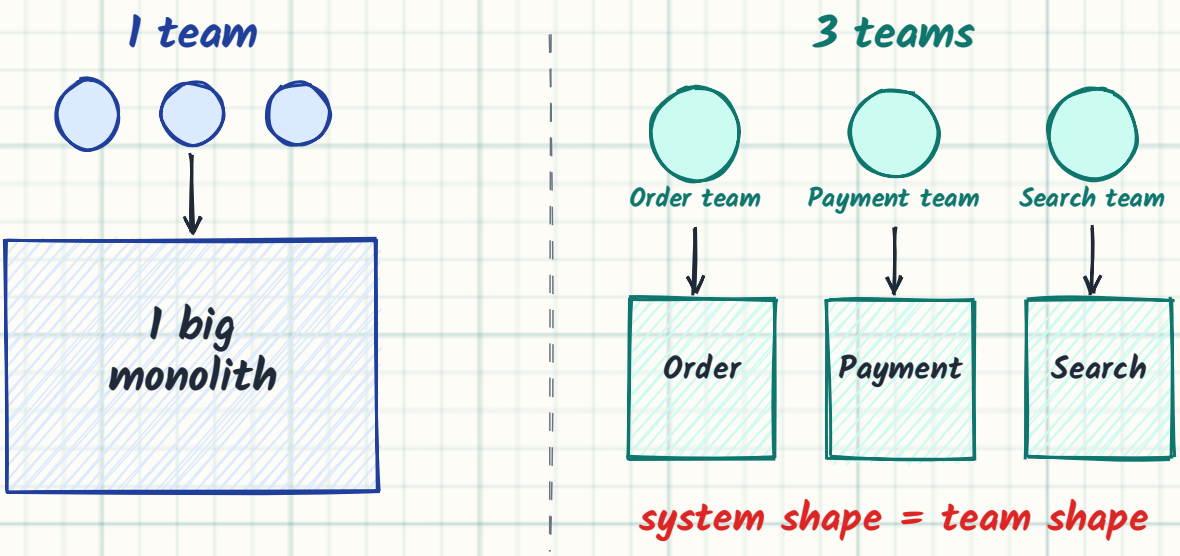
② The 9 characteristics

#	Characteristic	ShopNest example
1	Componentization via services	Order service is released without touching Payment
2	Organized around business capabilities	Teams for Orders, Payments, Search (not UI / backend / DB)
3	Products, not projects	Payment team builds **and runs** its service for its whole life
4	Smart endpoints, dumb pipes	Logic in services; HTTP / message broker only carries messages
5	Decentralized governance	Search uses Elasticsearch, Payments uses a SQL database
6	Decentralized data management	Each service owns its own database (Lesson 10)
7	Infrastructure automation	CI/CD pipeline for every service (Lesson 21)
8	Design for failure	Timeouts, retries, fallbacks; Netflix Chaos Monkey
9	Evolutionary design	Recommendation service can be rewritten on its own

Remember!
In interviews, list a few and **explain 3 with examples**: business capabilities, decentralized data, design for failure.

③ Conway's law

> **Conway's law** (Melvin Conway, 1967): organizations design systems that mirror their own communication structure.



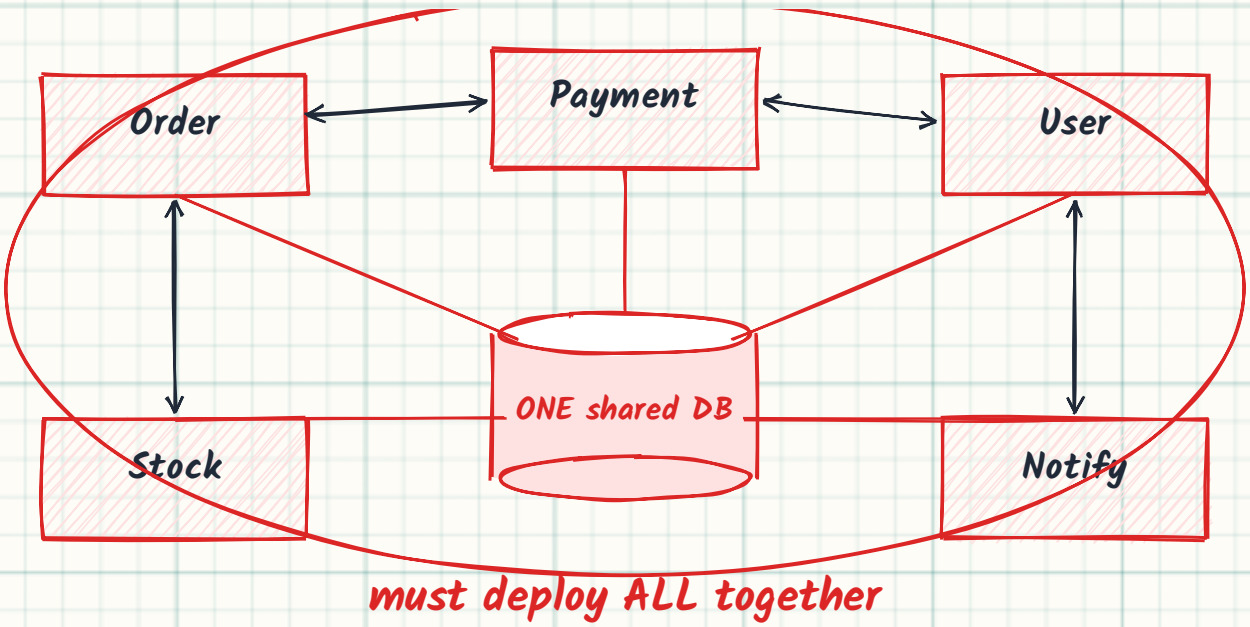
→ 1 team working together → tends to build **1 app**
→ 3 independent teams → tend to build **3 services**

Inverse Conway
Design the **team structure** you want, and the architecture follows. Give each team clear ownership.

Why it matters → if one team owns 12 services, or 5 teams share 1 service, microservices lose their main benefit: **independent teams moving fast**.

④ The distributed monolith trap

> **Distributed monolith** = services that are split physically, but so tightly coupled that they must be built, tested and deployed together.



Warning signs

- ✗ A release needs many services deployed **together**
- ✗ Services share **one database** or read each other's tables
- ✗ Long **synchronous call chains** (A → B → C → D)
- ✗ One feature change touches **5 services**
- ✗ Shared libraries full of **business logic**

Result
All the **costs** of microservices (network, ops, debugging) with **none of the benefits**.

⑤ 5 signs you should NOT use microservices

Sign	Why it hurts	Do instead
Small team (1–2 teams)	The team-coordination problem microservices solve doesn't exist yet	Modular monolith
Domain not clear yet	Requirements change weekly → boundaries drawn in the wrong place	Learn the domain first (Lesson 3)
No automation	No CI/CD, monitoring or containers → running many services by hand	Build the platform first
No real scaling need	Every part gets similar, modest load → independent scaling adds nothing	Scale the monolith (more instances)
Tightly connected data	Most features need one transaction across everything	Keep data together; revisit later

Rule of thumb
If 2 or more signs apply, **start with a modular monolith** and extract services only when there is a real reason.

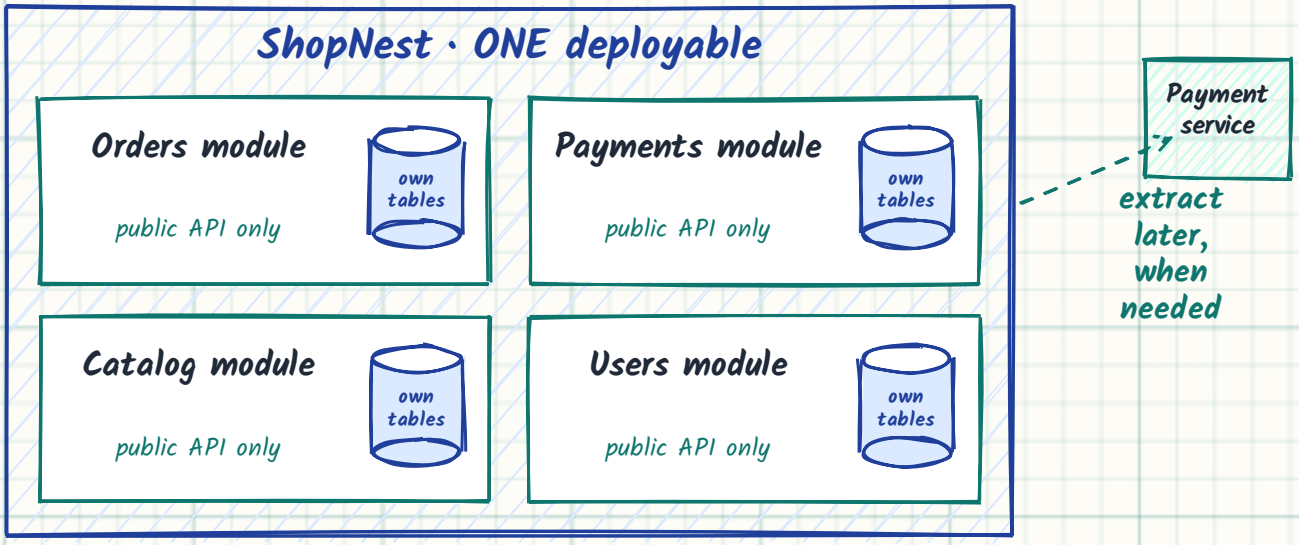
Quick self-check ✍️

1. How many teams will work on this system in the next year?
2. Can you name the business capabilities clearly today?
3. Can you deploy, monitor and roll back a service automatically?
4. Which part of the app actually needs to scale on its own?

Real example → Shopify is well known for running a very large **modular monolith**. Big scale does not automatically mean microservices.

6 Do this instead: the modular monolith

> **Modular monolith** = one deployable app, split inside into modules with strict boundaries: each owns its tables and exposes only a public API.



- ✓ Simple to deploy & debug (1 app)
- ✓ Boundaries already clean
- ✓ Extract a module into a service when it needs independent scaling or release

- → No module reads another's tables
- → Calls go through public interfaces
- → Enforce with package rules / ArchUnit tests

7 Readiness checklist

- ✓ Automated CI/CD pipeline per service
- ✓ Monitoring, logs & distributed tracing
- ✓ Clear team ownership per service
- ✓ Boundaries based on business capabilities

⑧ Interview Corner

Q. What are the characteristics of microservices?

A. The **9 from Lewis & Fowler**. Explain 3 with examples: business capabilities, decentralized data, design for failure.

Q. What is a distributed monolith?

A. Services split physically but coupled so tightly (shared DB, sync chains, shared logic) that they must deploy **together**.

Q. When would you NOT use microservices?

A. Small team, unclear domain, no automation, no scaling need, tightly connected data → prefer a **modular monolith**.

Q. What is Conway's law?

A. Systems mirror the communication structure of the organization that builds them.

Q. Smart endpoints, dumb pipes: meaning?

A. Business logic lives in the services; the network or broker only carries messages.

⚡ Revise in 60 seconds

- → **9 characteristics** · Lewis & Fowler 2014
- → Split by **business capability**
- → Each service **owns its data**
- → **Design for failure** + automation
- → **Conway**: system shape = team shape
- → Avoid the **distributed monolith**
- → **5 signs** → not ready
- → Start with a **modular monolith**

Next lesson → MS-03
Domain-Driven Design: **bounded contexts & aggregates**, how to find the right service boundaries.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**